

A Dynamic Logic for Verification of Synchronous Models based on Theorem Proving

Yuanrui ZHANG¹, Frédéric MALLET², Zhiming LIU (✉)^{1,3}

¹ School of Mathematics and Statistics, Southwest University, Chongqing, 400715, China

² University Cote d’Azur, CNRS, Inria, I3S, Nice, 06103, France

³ Center for Intelligent and Embedded Software, Northwest Polytechnical University, Xi’an, 710129, China

© Higher Education Press and Springer-Verlag Berlin Heidelberg 2012

1 Introduction

Synchronous models, a modelling paradigm proposed along with the development of synchronous programming languages [1] in 1980s, are well-adapted for modelling and specifying reactive systems — a type of systems that consecutively interact with their environment at a particular environment-determined rate. In a synchronous model, system behaviours are interpreted as a sequence of *reactions*. In each reaction, system inputs, data computations and system outputs are considered as being executed at the same time, but in some *logical precedence order* determined by their data dependencies. Such a discrete model of time correctly depicts the real-time nature of reactive systems. It also simplifies the modelling of concurrency, as all non-deterministic behaviours of a system are modelled as deterministic ones by treating all disordered, concurrent events as occurring simultaneously. Although these advantages make synchronous models popular as formal languages being used in both academics and industries, reasoning about synchronous programs has always been a challenge due to the complexity of their underlying synchronous execution mechanisms.

Dynamic logic [2] provides an intuitive framework for specifying and verifying program properties in a semi-automatic theorem-proving approach. It carries both programs and formulas as its parts, in a form of $[p]\phi$, called a *dynamic formula*. $[p]\phi$ expresses that after all executions of program p , formula ϕ is satisfied. $[\cdot]$ is a modality, originally from modal logic. Dynamic formulas allow reasoning about programs in a Floyd-Hoare style, in which a dynamic formula is transformed step by step into pure logical formulas without

programs (also called “verification conditions”) by disposing programs according to their syntactic structures. Traditional first-order dynamic logic (FODL) [3] does not support synchronous execution mechanism and cannot specify temporal properties, which are crucial for reactive systems.

In this paper, we propose a variation of dynamic logic, called *synchronous dynamic logic* (SDL), for reasoning about synchronous programs. SDL is based on FODL, but is extended with the notion of reactions and parallel operator to model synchronous execution mechanism and synchronous communications, and with temporal dynamic formulas from [4] to express safety properties. We build a sound proof system for SDL, which is also proved to be complete relative to arithmetical first-order logic (AFOL) under a certain condition. We give a simple example to illustrate the usage of SDL in practise.

In Online Resource 1 we list some existing related works about verifying synchronous models using different techniques, such as model checking, term rewriting, equivalent checking and theorem proving. There we make a rough comparison between them and our proposed logic.

2 Synchronous Dynamic Logic (SDL)

The syntax of SDL is given by simultaneous induction in the following definitions of programs and formulas. The programs of SDL, called *synchronous programs* (SP), is defined as follows in BNF form:

$$p ::= \mathbf{1} \mid \mathbf{0} \mid \alpha \mid p; p \mid p \cup p \mid p^* \mid \cap(p, \dots, p),$$

where α is defined as:

$$\begin{aligned} \alpha &::= \epsilon \mid \text{evt} . \alpha, \\ \text{evt} &::= \psi? \mid \varrho? \mid \varsigma!e \mid x := e, \\ \varrho &::= \hat{\varsigma}(x) \mid \bar{\varsigma}. \end{aligned}$$

In SP we introduce *macro event* α , a collection of all events *evt* that occur in a reaction. In each macro event, events are executed in sequence, separated by $\cdot$. This reflects the logical precedence order between events during one reaction. We import the notions of *signals* $\varsigma!e$ and *signal conditions* $\varrho?$ in the regular programs of FODL. Similarly as synchronous programming languages such as Esterel [5], they are for communication between parallel programs. $\cap$ is the parallel operator. $\cap(p_1, \dots, p_n)$ means that n programs $p_1, \dots, p_n$ are running concurrently. Other primitives and operators come from FODL directly and Online Resource 1 gives a detailed introduction.

The syntax of an SDL formula is given as follows:

$$\phi ::= tt \mid \theta(e, e) \mid \neg\phi \mid \phi \wedge \phi \mid \forall x.\phi \mid [p]\phi \mid [p]\Box\phi,$$

where θ is a predicate relation and p is a closed SP. We extend FODL formulas with a temporal dynamic formula of the form $[p]\Box\phi$ to capture safety properties. Formula $[p]\Box\phi$ means that ϕ holds at each reaction of the program p .

The semantics of an SP is defined as the set of its execution traces, defined as Definition 3.12 of Online Resource 1. The most different part from the semantics of FODL is to define the semantics of a macro event α and the semantics of a parallel program $\cap(p_1, \dots, p_n)$. In SDL, a macro event α is defined as a set of traces of the form ss' . s, s' represent the states before and after the execution of α , which relies on the executions of all events in α .

The most challenge part of defining the semantics of SPs is defining the semantics of a parallel program $\cap(p_1, \dots, p_n)$. When n programs $p_1, \dots, p_n$ are running in parallel, their events are executed simultaneously in one reaction. A *logical coherent* semantics is the one satisfying that in each reaction, each program p_i holds the same view towards the state of a signal. A *constructive semantics* is a logical coherent one plus that it also preserves the precedence order between all events in each reaction. As in Esterel [5], we need to define a constructive semantics for parallel SPs.

To achieve this, we propose a similar approach as proposed in [5] for Esterel. We define a function *Mer* to capture the communication process between concurrent programs $p_1, \dots, p_n$. *Mer* guarantees that the communication behaviour between $p_1, \dots, p_n$ is constructive in the above sense. This fact is formally stated in Proposition 3.2 of Online Resource 1, and there we also give the main idea of how to prove this proposition.

All the details of defining the semantics of parallel SPs is stated in Section 3.2.2 of Online Resource 1.

Based on the semantics of SPs, the semantics of SDL formulas is defined in Definition 3.22 of Online Resource 1.

3 Proof System of SDL

We propose a sound proof system for SDL in order to support verification of synchronous programs. All rules of the proof

system are given in Table 1-4 of Online Resource 1. In Table 1 we list all rules that are special for SDL, other rules are traditional rules for first-order logic.

The upper part of Table 1 are all rules for sequential SPs, which are extended from those in FODL with rules for special primitives in SPs. These rules are used for decomposing an SP into its subprograms. For example, in rule $(; \phi)$, in order to prove $[p; q]\phi$, we only need to prove $[p][q]\phi$, where p and q are subprograms of $p; q$ so the structure of the formula $[p][q]\phi$ is simpler than $[p; q]\phi$. The rules with a double middle line hold in both directions. For example, rule $(; \phi)$ actually means that $[p][q]\phi$ is valid iff $[p; q]\phi$ is valid. All rules are not hard to understand according to the semantics of SDL formulas, and they are fully illustrated in Section 4.2 of Online Resource 1.

The lower part of Table 1 shows the rules for parallel SPs. As SP is deterministic, the size of a parallel SP does not grow exponentially when transformed into a sequential one. We propose rewrite rules to reduce a parallel SP into a sequential one. Rule $(r1)$ and $(r2)$ are structural rules for rewriting an SDL formula or an SP according to the rewrites of its parts. Rule $(r1)$ says that if p can be rewritten by q , then p can be replaced by q anywhere of ϕ without the semantics of ϕ being changed. Rule $(r2)$ says that if q can be rewritten by r , then a program p where q is one of its subprograms can be rewritten to a program where any occurrence of q is replaced by r . Other rules are rewrite rules for different structures of parallel SPs. Section 4.3 of Online Resource 1 gives a full illustration of them.

In Section 5 of Online Resource 1, we prove the soundness of all rules in the proof system of SDL, and prove that SDL is complete relative to AFOL under a certain condition, that SPs must be “non-blocked”. Intuitively, an SP is non-blocked if it causes no deadlocks during its executions. Definition 5.1 formally describes this property. We give an explanation on why SDL is not relatively complete if an SP can be blocked. The skeleton of proving the relative completeness of SDL mainly follows the proof of FODL initially proposed in [3], but is augmented to adapt the new primitives and formulas in SDL. All details of the proofs are displayed in the Appendix of Online Resource 1.

4 A Case Study

To show the potential of our proposed SDL framework, we illustrate how SDL can be used to specify and verify syncChart models. syncChart is one of synchronous models that has the same semantics as Esterel and has been embedded into the famous industrial tool SCADE for practical usage. syncChart captures the most essential features of synchronous models and is in an automata form, which is easy to be transformed into SPs.

In Section 6 of Online Resource 1, we consider two examples of syncCharts, one of them is called “Frequency Di-

Table 1: Partial Rules for the Proof System of SDL

Fig. 1: A Frequency Divider

As for future work, we mainly focus on mechanizing SDL in the theorem prover Coq and apply SDL in specifying and verifying more interesting examples rather than the toy examples in this paper.

Fig. 2: The Derivation Tree of ϕ_{FD}

1. Benveniste A, Caspi P, Edwards S A, Halbwachs N, Le Guernic P, de Simone R. The synchronous languages 12 years later. Proceedings of the IEEE, 2003, 91 (1): 64 – 83
2. Harel D, Kozen D, Tiuryn J. Dynamic Logic. MIT Press, 2000
3. Harel D. First-Order Dynamic Logic. Lecture Notes in Computer Science (LNCS) 68, Springer, 1979
4. Platzer A. A temporal dynamic logic for verifying hybrid system invariants. In: Logical Foundations of Computer Science (LFCS), 2007: 457 – 471
5. Berry G. The constructive semantics of pure Esterel. <https://www-sop.inria.fr/members/Gerard.Berry/Papers/EsterelConstructiveBook.pdf>, 1999